CNN II Practicals

Structured Notes

Pooling Layers, CNN Building Blocks,

Full CNN Pipeline,
Parameter Counting, and Practical Implementation

Contents

1 Pooling Layers **2**
1.1 How Pooling Works . . . 2
1.2 Types of Pooling . . . 2
1.2.1 Max Pooling (Most Common) . . . 2
1.2.2 Average Pooling . . . 2
1.3 Why Max Pooling Is Preferred . . . 2
1.4 Pooling Has No Trainable Parameters . . . 3
1.5 Pooling in PyTorch . . . 3
1.6 Shape Flow Through Pooling . . . 3
1.7 Pooling on Multi-Channel Inputs . . . 3

2 CNN Building Block **4**
2.1 Role of Each Component . . . 4
2.2 Simple CNN Block Implementation . . . 4

3 What Makes CNN Special **5**
3.1 CNN vs ANN: Similarities and Differences . . . 5

4 Full CNN Pipeline for Image Classification **5**
4.1 Feature Learning vs Classification . . . 6
4.2 Component Roles in Full Pipeline . . . 6
4.3 Deep Learning vs Traditional Machine Learning . . . 6

5 Tensor Shape Convention: NCHW **6**
5.1 Default Parameter Values . . . 7

6 Parameter Count Formulas **7**
6.1 Conv2D Parameters . . . 7
6.2 Fully Connected Layer Parameters . . . 7
6.3 Comparison . . . 7

7 Practical CNN Implementation **8**
7.1 Two-Layer CNN Architecture . . . 8
7.2 Shape Flow Through the Network . . . 9
7.3 Output Size Computation Example (No Padding) . . . 9
7.4 Kernel Weight Initialization . . . 9
7.5 Why Increase Depth While Reducing Spatial Size . . . 9

8 Summary **9**

1 Pooling Layers

Pooling Layer

A **pooling layer** is a kernel with **no weights** (an empty kernel) used to reduce the spatial dimensions (height and width) of feature maps. Pooling does **not** reduce the depth (number of channels). It operates on each channel independently, reducing height and width while keeping the channel count constant.

CNN is analogous to ANN. Just as increasing the number of neurons increases dimensions in ANN, increasing the number of filters/kernels increases the depth in CNN. Once the depth increases, the number of parameters increases. Pooling is used to get the optimized output from this increased depth by reducing the spatial dimensions.

1.1 How Pooling Works

A standard pooling operation uses a 2×2 kernel with stride 2:

- Input shape: (C, H, W)
- Output shape: $(C, H/2, W/2)$
- The depth/channels C remain unchanged
- Only height and width are halved

1.2 Types of Pooling

1.2.1 Max Pooling (Most Common)

Max Pooling

Max pooling takes the **maximum value** from each patch covered by the pooling kernel. For a 2×2 max pooling with stride 2, it divides the feature map into 2×2 blocks and selects the maximum value from each block.

Example: Given a 4×4 feature map, applying 2×2 max pooling with stride 2:

- The kernel covers the first 2×2 patch, takes the maximum value
- Moves by stride 2 to the next patch, takes the maximum
- Continues until the entire feature map is processed
- Result: a 2×2 output containing the maximum values from each patch

Max pooling keeps the **strongest activation** from each region.

1.2.2 Average Pooling

Average Pooling

Average pooling computes the **mean** of all values in each patch. For a patch with values $\{-1, 2, 0, 4\}$, the average pooling output is $(-1 + 2 + 0 + 4)/4 = 1.25$.

In average pooling, the result tends to be biased towards the maximum value. Custom pooling operations using median or mode are also possible.

1.3 Why Max Pooling Is Preferred

Max pooling is preferred because:

- It extracts the **most important features** (highest activations) from each patch
- It provides **translational tolerance**—if a strong activation (e.g., an edge) shifts slightly within a patch, max pooling still captures it

- The maximum value often contains the information of surrounding features and represents the strongest signal

Information Loss in Pooling

There is 100% a loss of information during pooling. However, this loss can be traded off because:

- The most important features (highest activations) are retained
- First, information is increased through convolution (feature extraction), then optimized through pooling
- With the original number of parameters, computation would be too expensive

If max pooling is not performing well (important features are being lost), average pooling can be used as an alternative.

1.4 Pooling Has No Trainable Parameters

Key Interview Point

Pooling does not add any trainable parameters to the network. The pooling kernel has no weights. It only reduces features/parameters but does not add any trainable weights. Only convolutional kernels carry weights that can be trained. This is a common interview trap—many candidates incorrectly assume pooling has learnable parameters.

Pooling is essentially a **convolution operation without weights**—it is just used to reduce the height and width by extracting the maximum, average, or another aggregate value.

1.5 Pooling in PyTorch

Max Pooling Example

```
import torch.nn as nn

# Define max pooling: kernel_size=2, stride=2
pool = nn.MaxPool2d(kernel_size=2, stride=2)

# Before pooling: shape (1, 3, 32, 32)
# After pooling:  shape (1, 3, 16, 16)
```

Listing 1: Max Pooling in PyTorch

1.6 Shape Flow Through Pooling

For a 2×2 pooling with stride 2:

$$(N, C, H, W) \to (N, C, H/2, W/2)$$

Pooling does **not** change the number of channels. It only reduces spatial dimensions (height and width).

Example: If input is $(8, 16, 64, 64)$, after 2×2 pooling the output is $(8, 16, 32, 32)$.

1.7 Pooling on Multi-Channel Inputs

When pooling is applied to a multi-channel tensor (e.g., 3 RGB channels or 16 feature map channels), each channel is processed **independently**. The maximum value of each patch is found per channel. The depth does not increase or decrease—for every channel, the spatial dimensions are reduced independently.

2 CNN Building Block

CNN Building Block

The fundamental building block of a CNN consists of three operations in sequence:

$$\textbf{Convolution} \rightarrow \textbf{ReLU} \rightarrow \textbf{Pooling}$$

This block is repeated N times with different parameters to create deep CNN architectures.

2.1 Role of Each Component

Component	Operation	Effect
Convolution	Linear transformation	Detects patterns, increases depth
ReLU	Non-linear activation	Introduces non-linearity, enables complex feature learning
Pooling	Spatial reduction	Reduces spatial size, improves computational efficiency

Why ReLU After Convolution

Convolution is a **linear operation**—it does not introduce non-linearity by itself. ReLU (or any activation function) is required to enable **complex feature learning**. Without non-linear activation, stacking multiple convolution layers would be equivalent to a single linear transformation.

What If We Remove Pooling

If pooling is removed, the spatial size remains large, which directly increases computation requirements and memory usage. Using small convolution kernels followed by pooling is more optimized than using large kernels. Small filters capture more information, and pooling then selects the best features from that rich information. Using large kernels alone only performs linear transformation and misses rich features.

2.2 Simple CNN Block Implementation

Simple CNN Block

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNNBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(
            in_channels=3, out_channels=8,
            kernel_size=3, stride=1, padding=1
        )
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.conv(x)        # Linear transformation
        x = F.relu(x)           # Non-linearity
```

```
        x = self.pool(x)        # Spatial reduction
        return x

# Input:  (1, 3, 32, 32)
# After conv (padding=1): (1, 8, 32, 32)
# After ReLU: (1, 8, 32, 32) - no shape change
# After pool: (1, 8, 16, 16) - spatial halved
```

Listing 2: Single CNN Block in PyTorch

Observation after this block: depth increased ($3 \to 8$), spatial size reduced ($32 \to 16$), non-linearity introduced.

3 What Makes CNN Special

Key Properties of CNN

Four properties that make CNN special compared to standard ANN:

1. **Spatial locality** — Operates on small patches (local connections, not global)
2. **Weight sharing** — Same kernel is reused across all spatial locations
3. **Parameter efficiency** — Fewer parameters compared to fully connected layers
4. **Hierarchical feature extraction** — Early layers detect edges, deeper layers detect complex shapes

3.1 CNN vs ANN: Similarities and Differences

Similarities:

- CNN is still a linear + nonlinear operation
- Uses gradient-based learning (backpropagation)
- CNN is a structured ANN

Key Difference: Convolution is a **linear transformation with structured sparsity plus weight sharing**. Connections are **local, not global**. This is why CNN is more parameter efficient than ANN on images.

CNN is not magic—it is just a structured ANN where the core idea is **respecting spatial geometry**.

4 Full CNN Pipeline for Image Classification

CNN Classification Pipeline

A standard CNN classifier follows this pipeline:

$$\text{Input} \to \underbrace{[\text{Conv} \to \text{ReLU} \to \text{Pool}]}_{\text{Feature Learning}} \times N \to \text{Flatten} \to \underbrace{\text{FC Layers}}_{\text{Classification}} \to \text{Output}$$

The convolution blocks handle **feature learning** (automatic feature engineering), while the fully connected layers perform **classification**.

4.1 Feature Learning vs Classification

Stage	Components	Purpose
Feature Learning	Conv → ReLU → Pool blocks	Automatic feature extraction
Flatten	Reshape 3D tensor to 1D vector	Prepare for classification
Classification	Fully connected layers	Map features to class probabilities

4.2 Component Roles in Full Pipeline

- **Convolution** — Extracts local spatial features
- **ReLU** — Introduces non-linearity
- **Pooling** — Reduces spatial resolution
- **Flatten** — Converts feature maps to vectors
- **Fully Connected Layer** — Performs classification (also called dense layers)

The model returns **logits** (raw predicted values). Logits are then converted to probabilities using **softmax** (multi-class) or **sigmoid** (binary). The process is: first prediction (logits), then classification (probability assignment).

4.3 Deep Learning vs Traditional Machine Learning

Key Interview Point

The main benefit of deep learning over traditional machine learning is **automatic feature engineering**. In traditional ML, features must be engineered manually. In deep learning, the convolution blocks automatically learn and extract features.

Furthermore, the CNN feature extraction blocks can be combined with traditional classifiers (SVM, logistic regression) instead of fully connected layers. This approach is called a **hybrid model**.

5 Tensor Shape Convention: NCHW

CNN tensors follow the **NCHW** format:

$$X \in \mathbb{R}^{N \times C \times H \times W}$$

where:

- N = Batch size (number of data points/images)
- C = Number of channels
- H = Height
- W = Width

For a convolution layer, the parameters are: input channels C_{in}, output channels C_{out}, kernel size K, stride S, and padding P.

Output spatial dimensions:

$$H_{\text{out}} = \frac{H + 2P - K}{S} + 1, \qquad W_{\text{out}} = \frac{W + 2P - K}{S} + 1$$

Output tensor shape: $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$

5.1 Default Parameter Values

Parameter	Default Value	Effect
Out channels	1 (if not defined)	Single filter used
Stride	1	Kernel moves one step at a time
Padding	0	No zero-padding added

If `out_channels` is not defined and only a single kernel is given, the default output depth is 1. To use multiple filters, the output channel count must be explicitly defined.

6 Parameter Count Formulas

6.1 Conv2D Parameters

For a 2D convolutional layer:

One kernel: $C_{\text{in}} \times K^2 + 1$ parameters (weights + 1 bias)

Each kernel has the same depth as the input image (C_{in}). The kernel spatial size is $K \times K$, giving $C_{\text{in}} \times K^2$ weights, plus 1 bias term.

Total parameters:

$$\text{Parameters} = C_{\text{out}} \times (C_{\text{in}} \times K^2 + 1)$$

Multiply the single kernel parameter count by the total number of kernels (C_{out}).

CNN Efficiency

Convolution parameter count does not depend on image height or width. It depends only on the kernel size, input channels, and output channels. This is the **key source of CNN efficiency**—parameters do not explode as image size increases.

6.2 Fully Connected Layer Parameters

For a fully connected (dense) layer:

$$\text{Parameters} = D_{\text{in}} \times D_{\text{out}} + D_{\text{out}}$$

Or equivalently: $D_{\text{out}} \times (D_{\text{in}} + 1)$

where D_{in} is the input dimension (number of features) and D_{out} is the number of neurons. Each neuron has D_{in} weights plus 1 bias.

Fully connected layers scale directly with input dimensions. This is why the first FC layer (after flattening) typically has the most parameters in a CNN.

6.3 Comparison

Property	Conv2D	Fully Connected
Depends on image size	No	Yes
Formula	$C_{\text{out}}(C_{\text{in}}K^2 + 1)$	$D_{\text{out}}(D_{\text{in}} + 1)$
Scales with	Kernel size, channels	Input dimensions

7 Practical CNN Implementation

7.1 Two-Layer CNN Architecture

Full CNN Implementation

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class FullCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Conv layer 1: 3 input channels -> 16 output
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3,
                               stride=1, padding=1)
        # Conv layer 2: 16 input channels -> 32 output
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3,
                               stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # FC layers: 32*8*8 -> 64 -> num_classes
        self.fc1 = nn.Linear(32 * 8 * 8, 64)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):
        # Block 1: Conv -> ReLU -> Pool
        x = self.pool(F.relu(self.conv1(x)))
        # Block 2: Conv -> ReLU -> Pool
        x = self.pool(F.relu(self.conv2(x)))
        # Flatten
        x = x.view(x.size(0), -1)
        # Classification
        x = F.relu(self.fc1(x))
        x = self.fc2(x)  # returns logits
        return x

model = FullCNN(num_classes=10)
dummy = torch.randn(4, 3, 32, 32)  # 4 RGB images, 32x32
output = model(dummy)
```

Listing 3: Two-Layer CNN for Image Classification

7.2 Shape Flow Through the Network

Layer	Operation	Output Shape
Input	—	$(4, 3, 32, 32)$
Conv1 (padding=1)	$3 \to 16$ channels	$(4, 16, 32, 32)$
ReLU	No shape change	$(4, 16, 32, 32)$
Pool1	$\div 2$	$(4, 16, 16, 16)$
Conv2 (padding=1)	$16 \to 32$ channels	$(4, 32, 16, 16)$
ReLU	No shape change	$(4, 32, 16, 16)$
Pool2	$\div 2$	$(4, 32, 8, 8)$
Flatten	Reshape	$(4, 2048)$
FC1	$2048 \to 64$	$(4, 64)$
FC2	$64 \to 10$	$(4, 10)$

Note: With `padding=1` and 3×3 kernels, spatial dimensions are preserved through convolution. If padding is changed to 0, the dimensions reduce after each convolution.

7.3 Output Size Computation Example (No Padding)

For an input of $1 \times 1 \times 32 \times 32$ with kernel size 4, stride 1, padding 0:

1. After Conv1: $32 - 4 + 1 = 29 \Rightarrow$ shape $(1, 1, 29, 29)$
2. After ReLU: same shape
3. After Pool (2×2, stride 2): $29/2 = 14.5 \to \lceil 14.5 \rceil = 15 \Rightarrow$ shape $(1, 1, 15, 15)$
4. After Conv2: $15 - 4 + 1 = 12 \Rightarrow$ shape $(1, 1, 12, 12)$
5. After ReLU: same shape
6. After Pool: $12/2 = 6 \Rightarrow$ shape $(1, 1, 6, 6)$

7.4 Kernel Weight Initialization

Initially, kernel weights are **predefined** with some initialization algorithm. These weights then get **tuned by backpropagation** during training.

- If initial values are a good guess (close to optimal), fewer epochs are needed
- If initial values are poor, more epochs are required
- Eventually, the model will converge regardless

Gradients can be tracked using **autograd** and the parameter function. If some parameters are getting stuck, the following can be adjusted: type of pooling, activation function, or learning rate.

7.5 Why Increase Depth While Reducing Spatial Size

- **Increasing depth:** Captures richer, more complex features
- **Reducing spatial size:** Controls computation, reduces memory usage, and focuses only on high-level information

8 Summary

- Pooling is a kernel with **no weights** that reduces spatial dimensions (height and width) while keeping channels constant
- Max pooling (most common) selects the maximum value from each patch; average pooling takes the mean
- Pooling provides **translational tolerance**—shifted activations are still captured
- Pooling **does not add any trainable parameters**

- The CNN building block is: Convolution → ReLU → Pooling
- CNN is special due to spatial locality, weight sharing, parameter efficiency, and hierarchical feature extraction
- The full CNN pipeline: [Conv → ReLU → Pool] $\times N$ → Flatten → FC Layers → Output
- CNN tensors use NCHW format: Batch size, Channels, Height, Width
- Conv2D parameters: $C_{\text{out}} \times (C_{\text{in}} \times K^2 + 1)$ — does not depend on image size
- FC parameters: $D_{\text{out}} \times (D_{\text{in}} + 1)$ — scales with input dimensions
- Deep learning advantage over traditional ML: automatic feature engineering in convolution blocks
- CNN feature blocks can be combined with traditional classifiers (SVM, logistic regression) for hybrid models